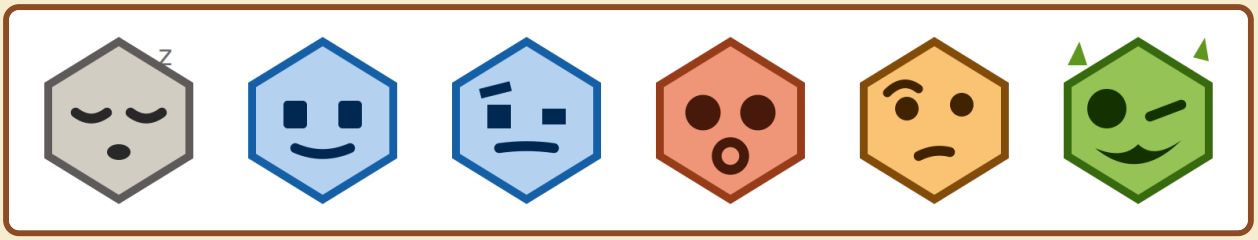
Isse Automate Kardo

Developer manual - end-to-end workings



Alexa+ MCP assistant · Python 3.12 · MCP spec 2025-11-25 · MIT licensed
Source: github.com/aashnology/Isse-Automate-Kardo
Written against the repository state of 16 October 2026. The project is still in progress; sections on limitations say what is and is not verified.

Contents

1 Overview

2 Quick start

3 The user journey and Hexi

4 Architecture

5 MCP tool reference

6 Algorithms

7 State and the consent model

8 Data adapters

9 The web layer, voice and extension

10 Security and hardening

11 Configuration

12 Testing and CI

13 Known limitations

14 References

A Repository layout

1 Overview

Isse Automate Kardo finds the repetitive workflows hiding in activity data, shows where the time goes and why, and proposes automating the low-judgment steps. Nothing runs until the user says yes. Its assistant, Hexi, answers by voice or text through a Model Context Protocol (MCP) server.

Design principles

- **Deterministic analysis.** Clustering, scoring, anomaly detection and even the spoken briefing involve no language model. The same input always yields the same answer, and every number can be traced to code in this repository.
- **Consent first.** Proposing and confirming are separate tools, called in separate turns. A single misheard word cannot execute anything.
- **Remember what was dismissed.** A standing preference is the one piece of state that persists to disk.
- **Two doors, one core.** An Alexa+ (or any MCP) client and the Hexi browser demo run the same functions.
- **Say what is untested.** Limitations are documented plainly, in this manual and in the README.

Hackathon context

Built for Amazon's *Build, Ship, Shape* hackathon, Alexa+ track, with an entry in the Open Source mini challenge. The Alexa+ track requires a self-hosted MCP server implementing spec 2025-11-25 or later over Streamable HTTP.

2 Quick start

Two services share one core. The MCP server alone is enough for any MCP client; Hexi adds the browser demo.

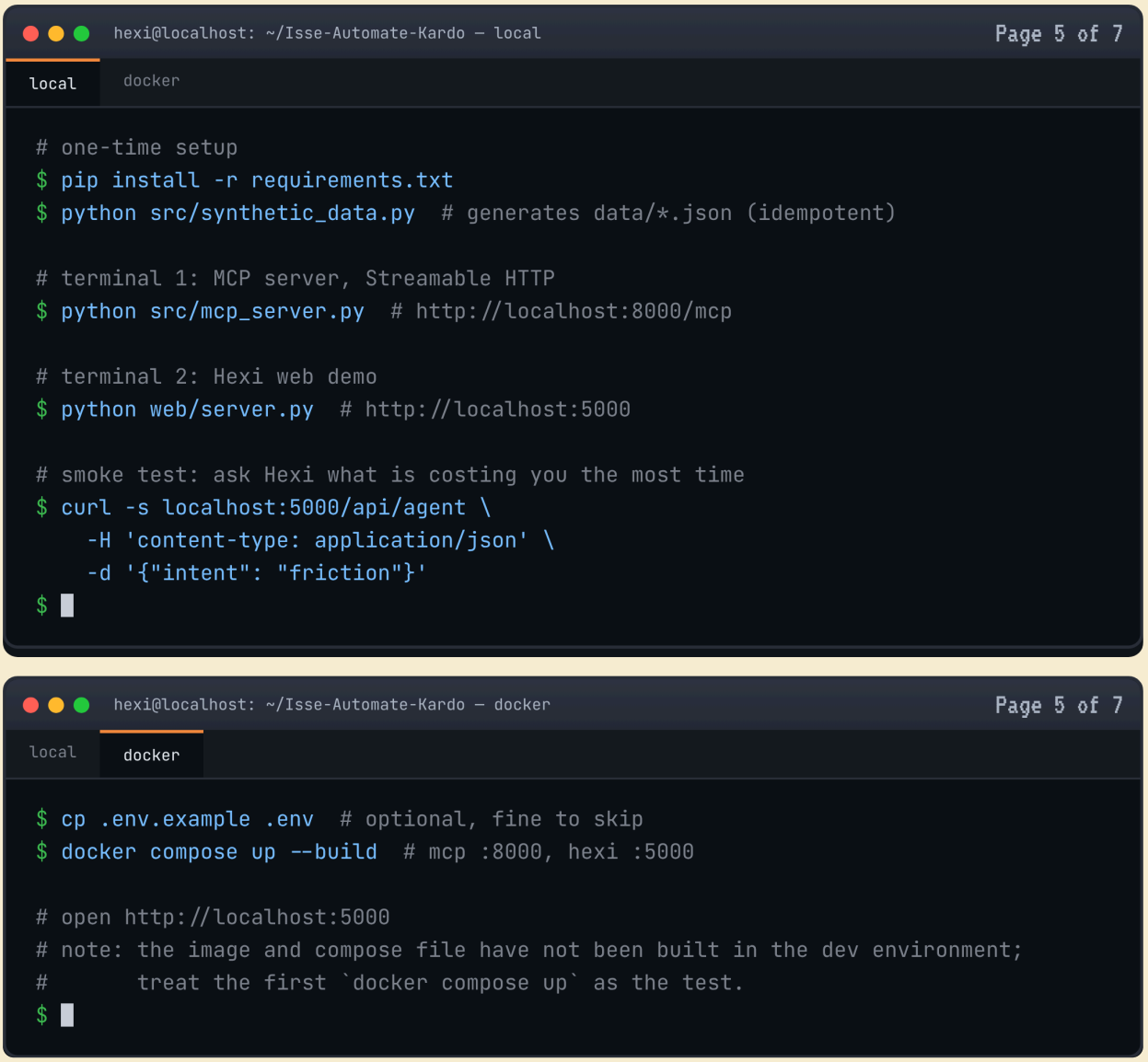


Figure 1. Local and Docker commands (README panel).

Local

```
pip install -r requirements.txt
python src/synthetic_data.py     # generates data/*.json (idempotent)
python src/mcp_server.py         # terminal 1: http://localhost:8000/mcp
python web/server.py             # terminal 2: http://localhost:5000
```

Docker

```
cp .env.example .env            # optional
docker compose up --build
```

> **Not yet verified.** The image and compose file have not been built or run in the environment this project was developed in. Treat the first `docker compose up` as the test.

The compose file defines two services. `mcp` binds `0.0.0.0` inside the container, which disables FastMCP's automatic DNS-rebinding protection, so it sets `MCP_ALLOWED_HOSTS` and `MCP_ALLOWED_ORIGINS` (the server refuses to start without them). `hexi` reaches it at `http://mcp:8000/mcp` via `HEXI_MCP_URL`. Both services bind to `127.0.0.1` on the host and share `./data`, so dismissals persist and both see one dataset.

3 The user journey and Hexi

The conversation

User says	Tool	Result
“What patterns are in my work?”	`discover_workflows`	Workflows clustered from raw event sequences, no pre-existing label.
“Where am I wasting time?”	`get_top_friction_points`	Workflows ranked by total time cost and automation tier.
“Why does bug triage take so long?”	`debug_workflow`	Step-level bottleneck breakdown.
“Was there anything unusual?”	`detect_anomalous_runs`	Outlier runs and their likely cause.
“Can you automate the reporting steps?”	`propose_automation`	Step-level proposal. Never executes.
“Yes.”	`confirm_automation`	Executes only after this explicit yes.
“Just give me the rundown.”	`narrate_briefing`	One spoken summary of computed findings.

Hexi's states

Hexi is a hexagonal character in the browser demo. Each expression reacts to a real call into the pipeline.

State	Trigger
asleep	The MCP server is not running.
idle	Server up; breathing, waiting.
watching	The user clicks through the simulated workspace or asks something.
curious	A real pattern appeared in the data.
suggesting	She is deciding whether the pattern is worth automating.
confirmed	The user said yes and the automation ran.

Hexi “watches” a sandboxed workspace built into the page. A website cannot see other tabs or sites, and the extension does not read page content, so no real cross-tab observation takes place.

4 Architecture

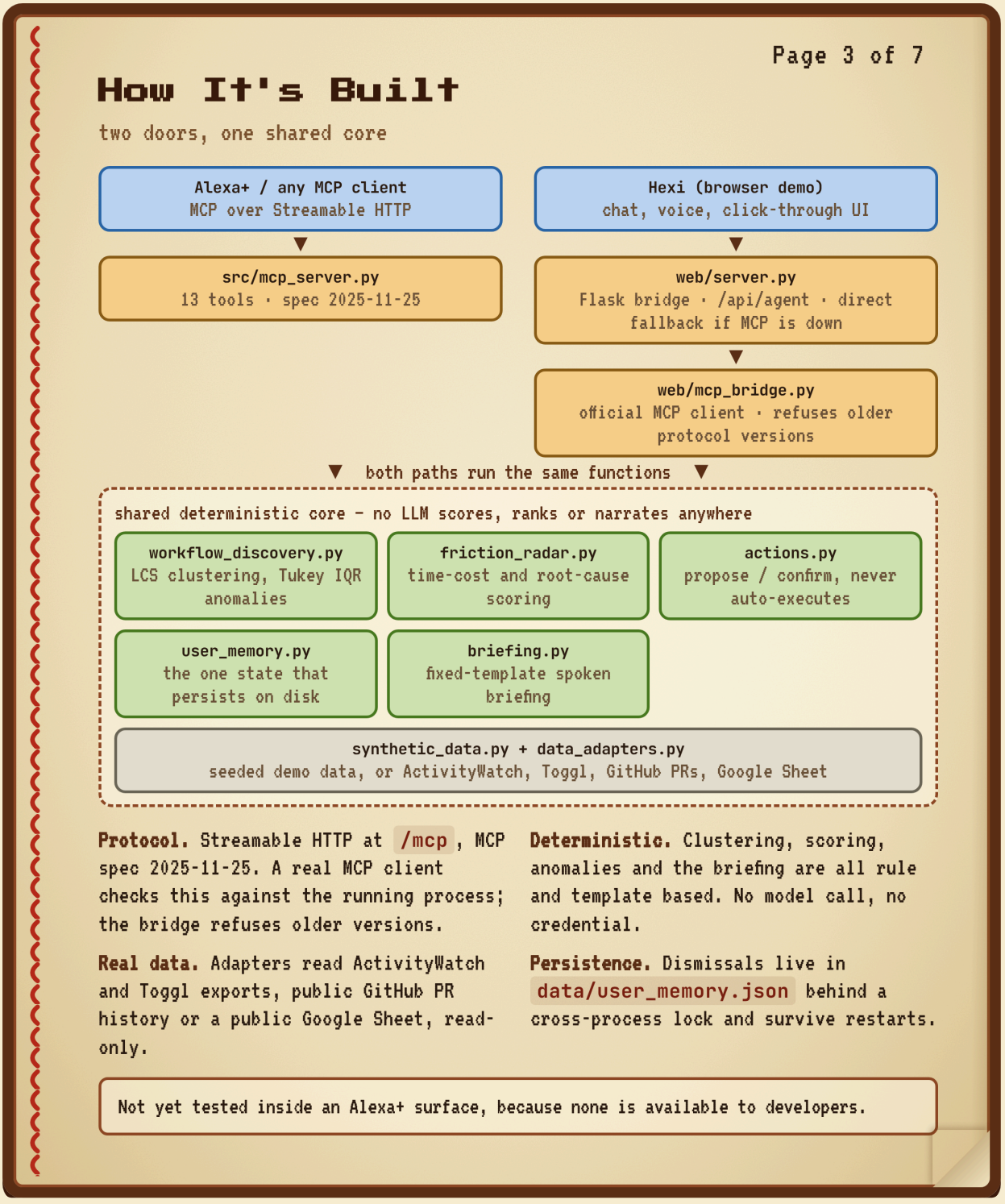


Figure 2. Two entry points over one shared deterministic core.

Note. An earlier diagram in the repository showed a `bedrock_narrator.py` module that does not exist. The briefing is built by `src/briefing.py`, a fixed template with no model call. The README now uses the corrected diagram above.

Components

Module	Role
`src/mcp_server.py`	FastMCP server (official `mcp` SDK, pinned 1.30.0), Streamable HTTP at `/mcp`, 13 tools with read-only / destructive / idempotent annotations.
`src/workflow_discovery.py`	Label-free clustering of runs into workflows; anomaly detection; `from_raw_events` for unsegmented streams.
`src/session_segmentation.py`	Reconstructs run boundaries from timestamps alone.
`src/friction_radar.py`	Frequency and time-cost analysis with an explainable automation-potential score. No LLM.
`src/actions.py`	Propose / confirm action layer with in-memory proposals.
`src/user_memory.py`	File-backed dismissals with a cross-process lock.
`src/briefing.py`	Single builder for the spoken briefing, shared by `narrate_briefing` and Hexi's “brief me”.
`src/data_adapters.py`	ActivityWatch, Toggl, GitHub PR history and Google Sheet adapters to the minimal event schema.
`src/safe_import.py`	Path allowlist for tools that take a file path.
`src/ranking_engine.py`	TF-IDF relevance, recency novelty and MMR diversity. Tested, deliberately not exposed as a tool.
`src/synthetic_data.py`	Seeded demo data, regenerated if missing.
`web/server.py`	Flask bridge: static page, `/api/agent`, import endpoints, rate limiting and security headers.
`web/mcp_bridge.py`	Official MCP client used by the web layer; refuses servers that negotiate an older protocol.
`web/aspen_bridge.py`	Starts and stops the optional Aspen companion.
`extension/`	Manifest V3 side panel that calls `/api/agent`.

Request flow for one spoken question

1. The browser transcribes speech; `web/static/intents.js` sanitises the text and classifies it to an intent.
2. `POST /api/agent` receives `{intent, workflow?, proposal_id?}`.
3. `web/server.py` calls the matching tool on the MCP server through `mcp_bridge.py`. If the MCP server is unreachable, it runs the same Python function directly and marks the response `via: "direct"`.

4. The reply, mood, tools used, negotiated protocol and any pending proposal id return to the page, which animates Hexi and speaks the reply locally.

5 MCP tool reference

Thirteen tools, verified end to end with a real MCP client in `tests/test_mcp_protocol.py`. Every tool carries MCP annotations; `openWorldHint` is false throughout.

Tool	Parameters	Hint	Behaviour
`discover_workflows`	—	read-only	Cluster the loaded event log into workflows by normalized-LCS sequence similarity.
`discover_workflows_from_import`	`source`, `file_path`	read-only	Same discovery over an ActivityWatch/Toggl export. The path is checked by `safe_import.py` before it is opened.
`get_top_friction_points`	`top_k=3`, `include_dismissed=False`	read-only	Rank workflows by total time cost and automation tier. Dismissed workflows are hidden unless requested.
`explain_friction`	`workflow_name`	read-only	Why a workflow ranks where it does, from the same score components.
`estimate_time_cost`	`workflow_name`	read-only	Time cost of one workflow.
`debug_workflow`	`workflow_name`	read-only	Step-level bottleneck breakdown.
`detect_anomalous_runs`	`workflow_name`, `top_k=3`	read-only	Runs outside the Tukey IQR fence for their own workflow, with the likely cause. Says so when there are too few runs.
`narrate_briefing`	`top_k=3`	read-only	One spoken answer built by a fixed template over computed results. Excludes dismissed workflows.
`propose_automation`	`workflow_name`	writes proposal	Returns a step-level proposal and a `proposal_id`. Executes nothing.
`confirm_automation`	`proposal_id`	destructive hint	Executes a matching proposal. Currently a stub (see Limitations).
`dismiss_workflow_suggestions`	`workflow_name`, `reason=""`	idempotent write	Persist a standing preference not to suggest a workflow.
`restore_workflow_suggestions`	`workflow_name`	idempotent write	Remove a dismissal.
`list_dismissed_workflow_suggestions`	—	read-only	List current dismissals.

6 Algorithms

Each method is implemented in this repository rather than wrapped from a library, so it can be read and explained line by line.

6.1 Workflow discovery

Given runs as ordered activity sequences with no workflow label, runs are grouped by pairwise sequence similarity. The metric is normalized Longest Common Subsequence (LCS), chosen because it tolerates reordered, missing and extra steps between two runs of the “same” workflow, which exact-match grouping would split apart. The framing follows process mining (van der Aalst et al., 2004) and the trace-clustering line of work (Bose & van der Aalst, 2009).

6.2 Run segmentation

A raw event stream has no run ids. `session_segmentation.py` exploits the gap distribution: steps inside one run are minutes apart, while separate runs are hours or days apart. It sorts the gaps and splits at the largest jump, an unsupervised threshold with no boundaries given in advance. On this project's own data it recovers true run boundaries with an Adjusted Rand Index of **0.905** (`python tests/segmentation_eval.py`). Errors concentrate in workflows that occur only a handful of times a week, which can merge with whatever ran beside them. The approach assumes workflows do not interleave.

6.3 Anomalous runs

Within one discovered workflow, a run is flagged when its duration falls outside Tukey's IQR fence. IQR is used instead of a z-score because durations are right-skewed: most runs sit near the median and a few blow far past it, and IQR fences do not assume symmetry. With only one or two runs the tool states there is too little data rather than guessing.

6.4 Friction scoring

`friction_radar.py` aggregates sequences into frequency and total time cost per workflow and combines them into an explainable automation-potential score, with an automation tier. Steps considered safe to propose for automation are those with low judgment cost, such as `column_cleanup`, `rename`, `export`, `upload`, `file_download` and `account_create`.

6.5 Diverse ranking (not exposed)

`ranking_engine.py` uses TF-IDF similarity, a recency-decay novelty score and Maximal Marginal Relevance (Carbonell & Goldstein, 1998) so a top-k list does not repeat one story. It is tested and evaluated (`python tests/evaluation.py`) but intentionally not part of the current tool surface.

6.6 Briefing

`briefing.py` over-fetches top friction points, drops dismissed workflows, attaches the strongest anomaly per workflow, and renders one sentence from a fixed template. Excluding dismissals here keeps a briefing from raising something the user asked never to hear about.

7 State and the consent model

Proposals (short-lived, in memory)

`propose_automation` stores a proposal under a generated id and returns it. Nothing executes. `confirm_automation(proposal_id)` executes only a matching, unexpired proposal. Proposals live in a lock-guarded in-process dict, capped at 1,000 entries, and expire after one hour, so an old “yes” cannot confirm a stale proposal. The Agent Skill and the web voice flow add a second guard: propose and confirm happen on separate turns.

Dismissals (durable, on disk)

`user_memory.py` stores standing preferences in `data/user_memory.json`. The file is shared by two live processes, the MCP server and the Flask bridge, so every read-modify-write sits inside a cross-process lockfile created with `O_CREAT|O_EXCL` (atomic on POSIX and Windows, no third-party dependency). A lock older than 30 seconds is treated as a crashed owner and broken; waiting gives up after 10 seconds. Workflow names are capped at 200 characters and reasons at 500. A single file is deliberate: the deployment has one user and no multi-tenant story.

8 Data adapters

Adapters convert real export formats to the minimal event schema the discovery pipeline needs:

```
{"timestamp": "<ISO 8601>", "activity": "<str>", "duration_minutes": <float>}
```

Sources: ActivityWatch exports, Toggl Track exports, a public GitHub repository's pull-request history (read-only API calls; it never clones or runs code), and a public Google Sheet or Drive link. Pasted step lists are also accepted from the “My data” tab. Every path only reads and parses; nothing imported is executed.

Real exports carry no success or rework outcome, so rework-rate and judgment-cost scoring cannot run meaningfully on adapted data. What works end to end on real data is segmentation, then clustering, then time-cost ranking and anomaly detection.

9 The web layer, voice and extension

/api/agent

```
curl -s localhost:5000/api/agent -H 'content-type: application/json' \
  -d '{"intent": "friction"}'
```

Request: `{intent, workflow?, proposal_id?}`. Response: `reply`, `mood`, `tools`, `protocol`, `pending_proposal_id`, plus `via` (`mcp` or `direct`). Unrecognised input returns a normal reply listing what can be asked. Error statuses are 413 (oversized body), 503 (MCP server down) and 500 (unexpected failure), each with a `reply`.

Intent handling

`intents.js` strips markup and control characters, caps input at 300 characters and corrects small typos with one-edit matching on words of five or more letters sharing a first letter (so “grief” does not become “brief”). It is keyword matching, not language understanding.

Voice

Input uses the browser's speech recognition (Chrome, Edge, Safari); in Chrome and Edge the audio goes to the browser vendor's cloud service. Hexi's spoken replies are generated on the device.

Hands-free mode (simulated Alexa+)

The “Hey Alexa” toggle uses the laptop microphone as a stand-in for an Echo. Speech without the wake word is dropped; for a few seconds after Hexi speaks, a short reply such as “yes” is accepted. Wake-word matching is text matching on the transcript, not acoustic detection, and has not been run on a real Echo. Logic: `web/static/alexa.js`; tests: `tests/test_alexa_wake.py` (needs node).

Chrome extension

A Manifest V3 side panel that sends the same requests to `/api/agent` and shows each stage (heard, routed, request, MCP tool, reply). It talks only to localhost. Load it via `chrome://extensions` → Developer mode → Load unpacked → `extension/`. It keeps its own copy of `intents.js`; run `extension/sync.sh` after changes, and a test fails if the copies differ. Tested: manifest, file references and request logic in node, including against both live servers. Not tested: loading into Chrome, and side-panel microphone capture.

Aspen (optional)

A separate desktop companion in its own repository. After a confirmed automation Hexi asks whether to call him in; with `ASPEN_HOME` unset she says he is not set up and carries on. Hexi only starts and stops him.

10 Security and hardening

- **Loopback by default.** The MCP server binds `127.0.0.1`. Binding elsewhere requires `MCP_ALLOWED_HOSTS` and `MCP_ALLOWED_ORIGINS`, and the process refuses to start without them, so DNS-rebinding protection is never silently off.
- **Path safety.** `safe_import.py` resolves symlinks and requires the file to sit inside one allowlisted directory, have an allowed extension and be under a size cap. Rejections never echo the resolved path or contents.
- **Request limits.** The Flask layer caps bodies at 2 MB, rate-limits per client (60 requests per 60 seconds) and sets security headers including a Content-Security-Policy.
- **Input handling.** Client-side sanitising, escaping tests for the front end, and a rule in the Agent Skill that tool output is data, never instructions.
- **Non-root container.** The Docker image runs as a dedicated unprivileged user.
- **Tokens.** `GITHUB_TOKEN` is optional, read-only, and gated behind `GITHUB_TOKEN_PUBLIC_DEMO_OK` for public deployments. Never commit tokens.

11 Configuration

Variable	Default	Purpose
`MCP_SERVER_PORT`	8000	Port for the Streamable HTTP transport.
`MCP_BIND_HOST`	127.0.0.1	Bind address. Non-loopback needs the two allowlists.
`MCP_ALLOWED_HOSTS` / `MCP_ALLOWED_ORIGINS`	empty	Comma-separated allowlists for DNS-rebinding protection.
`WEB_PORT` / `WEB_BIND_HOST`	5000 / 127.0.0.1	Hexi web demo address.
`HEXI_MCP_URL`	local MCP URL	Where the web layer finds the MCP server (set by compose).
`GITHUB_TOKEN`, `GITHUB_TOKEN_PUBLIC_DEMO_OK`	empty, false	Optional; lifts GitHub's 60/hour unauthenticated limit for PR-history import.
`ASPEN_HOME`, `ASPEN_PYTHON`	empty	Optional Aspen companion location.

The server runs with none of these set.

12 Testing and CI

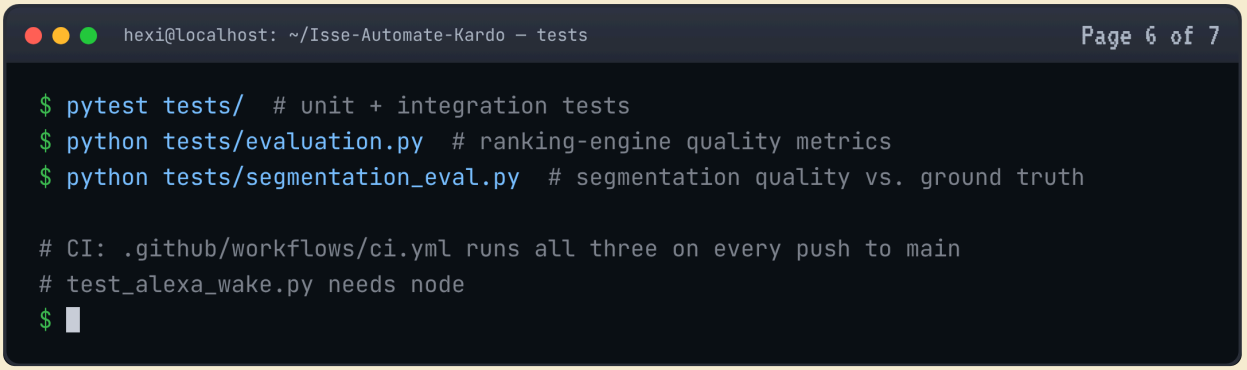


Figure 3. Test commands (README panel).

```
pytest tests/                    # unit + integration tests
python tests/evaluation.py       # ranking-engine quality metrics
python tests/segmentation_eval.py # segmentation vs. ground truth
```

The suite has 22 test modules and 168 test functions, covering the MCP protocol against the running server, tool annotations, the consent flow, proposal expiry and concurrency, user-memory persistence and locking, adapters, safe import, discovery limits, the web bridge, rate limits and headers, front-end escaping, intents, speech, wake word and the extension. Node is required for the wake-word and extension tests. GitHub Actions (`.github/workflows/ci.yml`) runs these on every push and pull request to `main` on Python 3.12.

13 Known limitations

Area	Limitation
Alexa+	Not tested inside an Alexa+ surface, since none is available to developers. Protocol conformance is verified with a real MCP client.
Hands-free	Wake word is text matching on a transcript; not run on a real Echo.
Confirm	`confirm_automation` is a clean stub. The propose-to-confirm flow works end to end, but execution is not wired to a real target system.
Segmentation	ARI 0.905 on this project's data. Rare workflows can merge with neighbours. Assumes workflows do not interleave.
Anomalies	Needs several runs of a workflow before saying anything.
Real data	No rework/outcome field in real exports, so judgment-cost scoring does not apply to them.
Google import	Not run against real Google servers; tested against a fake response. Try a real public sheet before relying on it.
Docker	Image and compose file not yet built or run.
Extension	Not loaded into Chrome in testing; side-panel microphone capture untested.
Intents	Keyword matching, not language understanding.
Scope	No view of other browser tabs; Hexi watches a sandboxed workspace only.

14 References

1. W. M. P. van der Aalst, A. J. M. M. Weijters, L. Maruster. *Workflow Mining: Discovering Process Models from Event Logs.* IEEE TKDE, 2004. doi:10.1109/TKDE.2004.47
2. R. P. J. C. Bose, W. M. P. van der Aalst. *Context Aware Trace Clustering.* SDM, 2009. doi:10.1137/1.9781611972795.35
3. J. Carbonell, J. Goldstein. *The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries.* SIGIR, 1998.
4. J. W. Tukey. *Exploratory Data Analysis*. Addison-Wesley, 1977.
5. Model Context Protocol specification, version 2025-11-25 (Streamable HTTP transport).

A Repository layout

```
.github/workflows/ci.yml   CI: pytest + evaluation scripts
data/                      generated demo data, user_memory.json
docs/                      notes, assets, this manual
extension/                 Manifest V3 side panel
skills/friction-radar/     Agent Skill (SKILL.md)
src/                       MCP server and deterministic core
tests/                     22 test modules + 2 evaluation scripts
web/                       Flask bridge, MCP bridge, static front end
Dockerfile, docker-compose.yml, requirements.txt, .env.example, LICENSE
```